

Redis 在软件项目中的应用

顾少伟, 井波

(江阴市大数据股份有限公司, 江苏 江阴 214400)

摘要: Redis是一个开源的内存数据结构存储系统, 被广泛应用于软件项目中。探讨了Redis在软件项目中的应用。首先, Redis可以作为缓存层来提高系统的性能和响应速度。通过将经常访问的数据存储在内存中, 可以大大减少对数据库的访问次数, 从而提高系统的效率。其次, Redis还可以用作消息队列, 用于实现异步通信和解耦系统组件。通过将消息存储在Redis中, 可以实现不同组件之间的松耦合, 提高系统的可扩展性和稳定性。再次, Redis还提供了丰富的数据结构和操作, 例如, 哈希表、列表、集合等, 可以用于解决各种实际问题, 例如, 计数器、排行榜等。最后, 讨论了Redis在使用过程中的一些问题。Redis在软件项目中的应用非常广泛, 可以提供高性能、可扩展和可靠的数据存储和处理功能。

关键词: Redis系统; 高并发; 非关系型数据库; 内存存储

DOI:10.16184/j.cnki.comprg.2023.11.046

1 概述

目前, 软件技术正处于高速发展的阶段, “高可用”已经成为一款优秀软件的标准。传统的关系型数据库渐渐无法满足日常的业务需求, 这就必须依靠非关系型数据库。Redis作为当下最流行的非关系型数据库, 对于它的熟练运用, 是每个软件开发人员必须掌握的一项技能。

2 Redis的原理

2.1 基于内存存储

Redis是一种基于内存存储原理的开源数据库系统。它使用内存作为主要的数据存储介质, 通过将数据存储在内存中, 提供快速的读写操作。与传统的磁盘存储系统相比, Redis具有更高的性能和低延迟。

2.2 持久化方式

Redis支持两种持久化方式, 即RDB (Redis Database) 和AOF (Append Only File)。

Redis的RDB持久化方式是一种将数据快照保存到磁盘的方法。RDB持久化方式通过将Redis内存中的数据以二进制格式写入硬盘中的RDB文件中来实现数据的持久化。这种方式可以在Redis重启时, 通过将RDB文件重新加载到内存中, 恢复数据。RDB持久化方式具有快速且紧凑的特点, 适用于备份、灾难恢复和数据迁移等场景。同时, RDB持久化方式也支持设置自动触发快照保存的策略, 可以根据时间间隔和数据变化量来进行配置。

Redis有两种AOF持久化方式: always和everysec。

always表示每个Redis命令都会立即被写入AOF文件中, 确保了最高级别的持久化, 但会带来较高的性能

开销。

everysec表示Redis每秒钟将AOF缓冲区中的命令写入AOF文件中, 这种方式在性能和持久化之间取得一个平衡。

可以通过配置文件中的“appendfsync”选项来选择使用哪种AOF持久化方式。在默认情况下, Redis使用的是“everysec”方式。

2.3 布隆过滤器

Redis布隆过滤器是一种数据结构, 用于快速判断一个元素是否存在于一个集合中。它基于哈希函数和位数组来实现, 可以在时间复杂度为 $O(1)$ 的情况下进行判断。布隆过滤器的特点是可以高效地判断一个元素是否存在于集合中, 但是在判断一个元素不存在时, 有一定的误判率。这是因为布隆过滤器使用多个哈希函数将元素映射到位数组中的多个位置, 如果某个位置的值为0, 则可以确定该元素一定不存在于集合中; 但是如果某个位置的值为1, 则无法确定该元素是否存在于集合中, 可能是对其他元素的误判。因此, 布隆过滤器适用于那些可以容忍一定误判率的场景, 例如, 缓存、网页爬虫等。

2.4 主从复制原理

在搭建Redis集群时, 采用主从复制策略, 具体如下: 当主从服务器刚建立连接时候, 进行全量同步, 全量复制结束后, 进行增量复制。当然, 如果有需要, 则slave在任何时候都可以发起全量同步。

作者简介: 顾少伟 (1992—), 男, 本科, 助理工程师, 研究方向为计算机软件开发。



master 服务器在内存缓冲区中给每个 slave 服务器都维护了一份同步备份日志 (in-memory backlog), 缓存最近一段时间的数据, 默认大小为 1 MB, 如果超过这个大小, 就会清理掉。

同时, master 和 slave 服务器都维护了一个复制偏移量 (replication offset) 和 master 线程 ID (master run id), 每个 slave 服务器在跟 master 服务器进行同步时都会携带 master run id 和最后一次同步的复制偏移量 offset, 通过 offset 可以知道主从之间数据不一致的情况。

当连接断开时, slave 服务器会重新连接上 master 服务器, 然后请求继续复制。假如主从服务器的两个 master run ID 相同, 并且指定的偏移量 offset 在同步备份日志中有效, 复制就会从上次中断的点开始继续。如果其中一个条件不满足, 就会进行完全重新同步, 因为主运行 ID 不保存在磁盘中, 所以如果从服务器重启, 就只能进行完全同步了。

2.5 高可用

Redis 提供了 Sentinel 和 Cluster 两种高可用方案。

Sentinel 是 Redis 官方提供的高可用性解决方案。它是一个分布式系统, 用于监控 Redis 节点的状态, 并在主节点故障时自动进行故障转移。Sentinel 可以配置多个哨兵节点, 它们会相互通信并监控 Redis 节点的状态, 当发现主节点故障时, 会选举新的主节点并更新配置。

Cluster 是 Redis 官方提供的分布式解决方案。它将数据分片存储在多个节点上, 每个节点负责存储部分数据。当某个节点故障时, 系统可以自动进行故障转移。Redis Cluster 还可以动态扩展, 可以添加或删除节点, 以适应不同的负载需求。

3 Redis 的特点

3.1 快速查询

Redis 是一个基于内存的数据存储系统, 具有非常高的读写速度。它使用了高效的数据结构和算法, 可以在微秒级别处理大量的请求。查询效率对比 (以 MySQL 为例) 如表 1 所示。

表 1 查询效率对比

单位: s

数据库	查询 10 次耗时	查询 100 次耗时
Redis	0.0016	0.0075
MySQL	0.0363	0.2919

由此可见, 查询数量越多, 相比于传统的关系型数据库, Redis 的效率越高, 这对于一个软件项目来说, 有了质的飞跃。

3.2 消息发布与订阅

Redis 提供了发布订阅的功能, 可以实现消息的发

布和订阅。发布订阅是一对多的关系, 需要有信息的发布者和消息的订阅者。

发布者: 提供某个内容或主题, 把内容信息发送给多个对此内容感兴趣的订阅者。

订阅者: 对某个内容感兴趣, 需要实时获取新的内容, 只要关注的内容有变化, 就能立即得到通知。

值得注意的是, 发布的消息没有持久化, 订阅者只能收到订阅后发布者发布的消息, 意思是订阅前的消息不能收到, 取消订阅后发的消息也收不到。

3.3 分布式存储

Redis 可以通过分片的方式实现数据的分布式存储。它可以将数据分散存储在多个节点上, 以提高系统的扩展性和性能。Redis 为了确保其数据的可靠性, 在分布式存储中通常需要对所有数据进行定时备份。常用的备份方式是基于主从架构, 通过将一个主节点的数据同步到多个从节点上来实现数据备份。在发生主节点故障时, 从节点可以顶替主节点的工作, 从而确保数据的可靠性和高可用性。Redis 哨兵模式如图 1 所示。

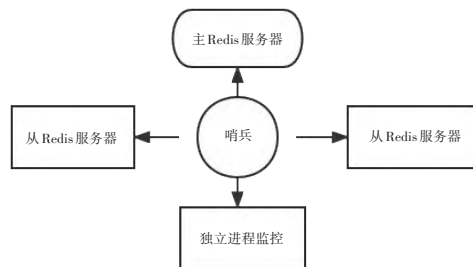


图 1 Redis 哨兵模式

4 Redis 的基础使用

(1) 安装 Redis。可以从 Redis 官方网站按照自己计算机的系统版本下载相应安装包并按照指南进行安装。

(2) 启动 Redis 服务器。安装完成后, 在命令行中输入 redis-server 命令来启动 Redis 服务器。

(3) 连接到 Redis 服务器。在另一个命令行窗口中, 使用 redis-cli 命令来连接 Redis 服务器。除此之外, 也可以通过一些第三方客户端进行连接, 例如, RESP、Redis Desktop Manager 等, 它们还提供了可视化页面, 以便进行操作。

(4) 存储和读取数据。使用 Redis 的 SET 命令可以将键值对存储到 Redis 中。例如, 使用以下命令将键名为 “mykey”, 值为 “value” 的数据存储到 Redis 中。

```
'''
SET mykey value
'''
```



(5) 使用 Redis 的 GET 命令可以读取存储在 Redis 中的数据。例如, 使用以下命令读取键名为 "mykey" 的数据。

```
GET mykey
```

(6) 使用其他 Redis 命令。除了 SET 和 GET 命令, Redis 还有很多其他命令可以用来操作数据。例如, 使用 DEL 命令可以删除指定的键值对, 使用 INCR 命令可以将指定键的值递增 1 等。可以查阅 Redis 官方文档以了解更多命令的使用方法。

(7) 关闭 Redis 服务器。在命令行中使用 “Ctrl+C” 组合键可以关闭 Redis 服务器。

(8) 以上这些是 Redis 的基础使用示例, 在实际运用过程中, 软件开发人员往往会根据自身使用的开发语言, 在 Redis 官网上找到适合的版本库, 从而进行对应的业务逻辑开发。

5 Redis 的适用场景

由于 Redis 具有高性能和低延迟的特点, 它被广泛应用于缓存系统。在许多 Web 应用中, 数据库查询是非常耗时的操作, 这样会严重影响用户体验。与此同时, 假如有成千上万条请求同一时间访问关系型数据库并进行增删改查操作, 则势必会给数据库服务器造成严重的压力, 万一处理不当就会使其崩溃, 导致软件系统的瘫痪。为了避免这种情况, 软件开发人员可以通过将查询结果缓存到 Redis 中, 让所有请求先在缓存中进行查询, 有结果就直接返回, 没有查到, 再到关系型数据库中进行查询。这样可以大大提升系统的性能, 提升处理请求的效率, 避免服务器因压力过大而崩溃。

Redis 还可以用作消息队列的中间件。Redis 提供了一种名为 “发布/订阅” 的模式, 可以用来实现消息队列功能。通过使用 Redis 的发布/订阅功能, 应用程序可以将消息发布到一个或多个频道, 其他应用程序可以订阅这些频道并接收消息。这种方式可以实现解耦和异步通信, 提高系统的可伸缩性和可靠性。消息队列模型如图 2 所示。

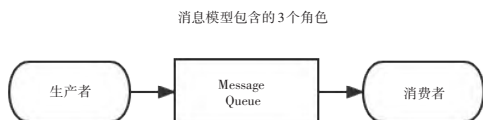


图 2 消息队列模型

此外, Redis 适用于计数器和排行榜等场景。由于其具有高速的读写能力, Redis 可以轻松处理大量并发操作, 从而实现实时计数和排名功能。

6 Redis 的应用案例

6.1 缓存功能

Redis 常用作缓存工具, 可以将常用的数据存储在 Redis 中, 以提高系统的读取速度。例如, 在电子商务网站中, 可以将商品信息、用户信息等存储在 Redis 中, 当用户请求时, 先从 Redis 中查询数据, 如果存在, 则直接返回, 减少了数据库的访问压力, 提高了系统的响应速度。以下是 Java 通过 Jedis 库读写 Redis 的参考代码。

```
java
import redis.clients.jedis.Jedis;
public class RedisExample {
    public static void main(String[] args) {
        // 连接 Redis 服务器
        Jedis jedis = new Jedis("localhost");
        // 写入数据
        jedis.set("key", "value");
        // 读取数据
        String value = jedis.get("key");
        System.out.println("Value: " + value);
        // 关闭连接
        jedis.close();
    }
}
```

该代码使用 Jedis 库来连接 Redis 服务器。首先, 创建一个 Jedis 对象并指定 Redis 服务器的地址 (本地地址为 “localhost”)。然后, 使用 ‘set’ 方法将一个键值对写入 Redis, 键为 “key”, 值为 “value”。使用 ‘get’ 方法读取键 ‘key’ 对应的值, 并将其打印输出。最后, 关闭连接。

运行这个示例代码, 需要在项目中添加 Jedis 库的依赖。可以在 Maven 项目中添加以下代码。

```
xml
<dependency>
    <groupId>redis.clients</groupId>
    <artifactId>jedis</artifactId>
    <version>3.6.0</version>
</dependency>
```

6.2 消息队列功能

Redis 的发布订阅功能使其可以作为消息队列的中

间件来实现异步通信。例如，在一个实时聊天应用中，可以使用 Redis 作为消息队列，当用户发送消息时，将消息发布到 Redis 中，然后其他用户通过订阅 Redis 的频道来接收消息。这样可以实现消息的实时传递，并且能够扩展到多个消费者，满足高并发的需求。

6.3 计数器功能

Redis 的原子操作特性使其非常适合用作计数器。例如，在社交媒体应用中，可以使用 Redis 来统计用户的“粉丝”数、点赞数、评论数等。通过 Redis 提供的 INCR 命令可以实现原子递增操作，保证计数的准确性，并能快速地进行计数操作，满足高并发的需求。

7 异常情况

通常 Redis 与关系型数据库搭配使用，在使用的过程中，可能会因使用不当而造成一些异常情况。下面以关系型数据库 MySQL 为例，列举了几个常见的 Redis 问题及解决方案。

7.1 缓存穿透

Redis 存储的数据基本以 key value 的形势存在，假如某个 key 对应的数据在 MySQL 数据库中并不存在，则如果每次针对此 key 的请求从 Redis 缓存获取不到，请求就会压到数据库 MySQL，从而可能压垮数据库。例如，用一个不存在的用户 ID 获取用户信息，无论是缓存还是数据库都没有，若黑客利用此漏洞进行攻击，则可能压垮数据库。解决方案如下：对空值缓存，如果一个查询返回的数据为空（不管是数据是否不存在），则仍然把这个空结果（null）进行缓存，空结果的过期时间会设置得很短，最长不超过 5 min。

7.2 缓存击穿

某个热点 key 对应的数据存在，但在 Redis 中过期，此时若有大量并发请求过来，则这些请求发现缓存过期后一般会从 MySQL 加载数据并回设到缓存，这时大并发的请求可能会瞬间把 MySQL 压垮。解决方案如下：（1）通过提前预埋热点数据，减少在高并发场景下接口直接访问 MySQL 的情况。（2）根据实际需求，调整 key 的过期时间。（3）设置线程锁，Java 通过 Jedis 设置线程锁的参考代码如下。

```
```java
Jedis jedis = new Jedis("localhost"); // 连接 Redis 服
//务器
// 获取锁
boolean lockAcquired = acquireLock(jedis);
if (lockAcquired) {
try {
```

```
// 执行需要加锁的操作
System.out.println("执行加锁操作");
Thread.sleep(5000); // 模拟执行加锁操作的耗时
} catch (InterruptedException e) {
e.printStackTrace();
} finally {
// 释放锁
releaseLock(jedis);
}
} else {
System.out.println("获取锁失败");
}
jedis.close(); // 关闭 Redis 连接
}
```
```

7.3 雪崩

雪崩的场景就是 Redis 中较多的 key 在同一时间全部失效，这时大量请求会直接落到 MySQL 上，造成数据库的压力过大，引发雪崩。解决方案如下：给项目设置多级缓存架构，例如，Nginx 缓存 + Redis 缓存 + 其他缓存（ehcache 等），从而降低 key 大量失效的概率；还可以将缓存失效的时间分散开来，例如，在原有的失效时间上，设置一个随机值，这样一来，每个缓存值的过期时间重复率会大大降低，集体失效的情况随之减少。

8 结语

对 Redis 进行了深入的研究和分析，并探讨了其在软件系统中的应用和重要性。通过对 Redis 的特点和功能进行详细介绍，不难发现 Redis 具有高性能、高可用性和灵活性的优势，可以满足各种不同规模和需求的应用场景。

通过对 Redis 的性能测试和比较，发现 Redis 在处理高并发读写请求时表现出色，并且具备快速响应和低延迟的特点。这使 Redis 成为了许多大型互联网公司和分布式系统的首选解决方案。

此外，还分析了 Redis 的数据模型和持久化机制，并讨论了其在缓存、消息队列等方面的应用。通过对针对这些应用场景的实际案例研究，验证了 Redis 在提升系统性能、降低数据库压力和保证数据一致性方面的优势。

综上所述，Redis 作为一种高性能的内存数据库，具有广泛的应用前景。但是，在实际应用中，开发人员需要根据具体的需求和场景，结合其他技术和工具来选（下转第 32 页）



5 结语

气象要素作为茶叶生产的资源和条件,直接影响着茶叶的产量和质量,为了细化茶叶气象服务,更好地为茶叶生产提供气象依据,增强茶园的防灾减灾能力,研究基于B/S架构开发了五莲茶园气象观测系统,采用数据库建设、Echarts技术对24 h降水、叶冠温度、土壤湿度等气象信息进行可视化显示,同时采用PHP语言设置定时任务获取茶园实景图片,实现了对茶叶长势情况及气象信息的实时监控,为茶叶的栽培、采摘等农事活动,提供了有效的气象依据,切实提高了当地茶叶的产量和质量。

参考文献

- [1] 张芬, 阎明升, 孙翠萍. 五莲县立足优势 大力发展近海山地茶[J]. 农业知识, 2021(18): 31-34.
- [2] 阎明升, 张芬, 孙翠萍. 徐经表的“茶样”人生——记五莲县“南茶北引”50年发展历程[J]. 农业知识, 2018(6): 39-41.
- [3] 白艳. 安吉白茶低温气象指数保险试点实践与思考[J]. 中国茶叶, 2019, 41(9): 57-59.
- [4] 陈荣冰. 茶树越冬芽萌发生长期与气象条件的关系[J]. 中国农业气象, 1987(3): 10-14.
- [5] 马于茗. 气象条件对浙江春茶生长及开采期的影响研究[D]. 南京: 南京信息工程大学, 2022.
- [6] 宋春燕, 刘春涛, 刘彬, 等. 影响崂山茶的气象因子及主要气象灾害的防御措施[J]. 农技服务, 2021, 38(7): 116-119.
- [7] 吴雪亚, 舒奕菲, 吴明, 等. 江口县茶叶生长气象条件分析及气象服务对策[J]. 现代农业科技, 2021(7): 42-43.

- [8] 钱半吨, 徐玉琴, 任义方, 等. 2021年气象条件对天目湖白茶茶叶产量和品质的影响分析[J]. 农业灾害研究, 2022, 12(7): 87-89.
- [9] 杨丽慧, 陈立, 白龙, 等. 安溪铁观音茶叶理化特征及其对气象因子的响应[J]. 海峡科学, 2022(1): 33-38.
- [10] 于晨. 福建省茶叶生产气象要素分析及气象保障服务研究[J]. 福建茶叶, 2021, 43(4): 21-22.
- [11] 高昕, 杨岗. 雅安市茶叶生产气象要素及气象灾害预防[J]. 农技服务, 2019, 36(11): 94-95.
- [12] 朱华松, 李波, 唐宁琳, 等. 紫金县茶叶种植气候适应性分析[J]. 气象研究与应用, 2018, 39(2): 59-62.
- [13] 王祺衡. B/S结构的软件开发技术研究[J]. 计算机产品与流通, 2019(11): 17.
- [14] 张虫金. 基于三层B/S模式的分布式系统设计[J]. 计算机产品与流通, 2020(10): 91.
- [15] 郑苏宁. Web跨站请求伪造的防护系统设计与实现[D]. 南京: 南京邮电大学, 2021.
- [16] 史良. 基于微服务架构的企业培训平台的研究与实现[D]. 北京: 北京工业大学, 2018.
- [17] 李继伟. 基于ASP.NET和Ajax技术的在线考试系统关键技术研究[J]. 电子测试, 2020(14): 74-75.
- [18] 任华, 邹承俊, 尹华国. 基于ASP.NET AJAX的在线购物系统的设计与实现[J]. 计算机与数字工程, 2013, 41(9): 1514-1516.
- [19] 明日科技. ASP.NET项目开发实战入门[M]. 长春: 吉林大学出版社, 2017.

(上接第19页)

择合适的解决方案。通过持续的研究和改进, Redis将在未来得到进一步发展和完善, 为分布式系统的构建和优化提供更多可能。

参考文献

- [1] 徐茂红, 王飞, 张明. 基于大数据量的Redis技术应用与研究[J]. 信息技术与信息化, 2019(11): 228-230.

- [2] 玉绍康, 关世杰, 苏一博. 基于Java环境下的Redis发布订阅的设计与实现[J]. 科技创新导报, 2019, 16(14): 3-4.
- [3] 张杰, 刘凯, 周立军. 采用Redis高并发应用系统设计与实现方法[J]. 信息科技, 2020(48): 1222-1226.
- [4] 张威. 基于redis的分布式缓存系统架构分析[J]. 计算机产品与流通, 2017(8): 252.

